Workshop 3

Pub/Sub Messages Documentation

Students:

Juan David Castaneda Cardenas
Laura Vanesa Alvarez Lafont
Nicolas Andres Bolanos Fernandez
Santiago Andres Fetecua Pulgarin
Sergio Nicolas Correa Escobar

Professor:

Carlos Andres Sierra Virguez

Software Engineering II
School of Engineering
Universidad Nacional de Colombia

This document describes the Pub/Sub message formats and topics that each microservice may asynchronously consume from **Kafka**, the selected broker.

Justification

The architecture of Fast Event Manager integrates two different communication styles. On one hand, REST is used for synchronous operations that require an immediate response from the frontend and for certain interactions between microservices. On the other hand, a Kafka-based Pub/Sub mechanism is employed for the propagation of domain events related to notifications, confirmations, and state changes within the system. This combination allows the platform to maintain a synchronous interaction model when immediacy is essential, and an asynchronous model when the priority is ensuring message delivery and loose coupling between components.

In a system where multiple components react to actions occurring in other modules (such as user registrations, attendance confirmations, waitlist promotions, or administrative alerts) it became clear that not all interactions should be synchronous. When a user registers, for example, the authentication microservice does not need to wait for the email to be sent in order to complete the request. What really matters is that the email is eventually delivered reliably, but the user experience should not depend on the state or availability of the email-sending service.

The Pub/Sub model provides exactly this separation between the component that produces an event and the one that processes it. Instead of coupling microservices through direct calls, each one simply publishes a message to a topic, without needing to know who will receive it or how many services will react to it. This approach allows the system to continue operating even if a service is temporarily offline, because the information is safely stored in the broker.

This communication style also simplifies development by avoiding direct dependencies between microservices whose interaction is not critical to request processing. The only shared interface between them is the structure of the messages they publish.

Thoughts on Observer

Initially, this notification workflow was understood in relation to the Observer design pattern. However, the traditional pattern only works in memory or inside a single application. Applying it across separate processes (and especially across services distributed over a network) would require implementing transport mechanisms, buffering, retries, fault tolerance, and message persistence.

The Pub/Sub model, by contrast, is designed specifically for distributed environments. It does not depend on observers being continuously available, nor does it require both ends to be active at the same time. Messages are not lost if a consumer is temporarily offline because the broker acts as a reliable intermediary.

Kafka, in this sense, provides the equivalent of a distributed Observer with memory and fault tolerance, adapted to the scale and requirements of the microservices-based project the team set out to learn.

Choosing Kafka

Among the many alternatives available for implementing a Pub/Sub system, Kafka stands out as an especially mature solution when high reliability is required. For this event manager, which the team attempted to develop as professionally as possible within learning conditions, Kafka offers a log-based distributed architecture with partitioning that allows it to handle large volumes of messages without compromising performance.

To better understand this decision, it is helpful to compare Kafka with other common options. RabbitMQ, for example, also offers a Pub/Sub model, but it is oriented toward more traditional queue-based patterns and flexible routing. While excellent for moderate workloads and more complex topologies, it tends to scale less effectively in systems with large, sustained event flows, such as those generated by high-concurrency event management platforms.

Cloud services like Google Pub/Sub or AWS SNS/SQS offer a similar model, but they rely on cloud infrastructure. Although they provide high availability and ease of use, they are not always ideal in local or academic environments, and their pricing models can be less predictable under supposed future fluctuating workloads. Furthermore, many of these services do not allow messages to be consumed multiple times nor provide a persistent log of historical events, which limits the ability to debug or recreate past scenarios.

Messaging Documentation

The topics that are created in this system are the following:

- **email.send:** Every microservice that requires to send an email publishes to this topic.

1. SignUp and LogIn

This microservice does not subscribe to any topic of the Pub/Sub server, but it publishes to email.send.

2. Event Management

This microservice does not subscribe to any topic of the Pub/Sub server, it does not publish to any either.

3. Confirmation

This microservice does not subscribe to any topic of the Pub/Sub server, but it publishes to email.send.

4. Email Notification

Every message sent to the email.send topic, which this microservice is subscribed to, must follow this JSON structure:

```JSON
{
  "to": "string",
  "subject": "string",
  "type": "EmailType",
  "params": { ... }
}
```

Field	Type	Required	Description
`to`	string	Yes	Recipient email address
`subject`	string	Yes	Email subject line
`type`	EmailType (enum)	Yes	Template to use
`params`	Map<string, object>	Yes	Template parameters (depend on the email type)

Depending on the type, the params must be different, as shown below.

a. SUCCESSFUL_REGISTER

Param	Type	Description

`name`	string	User's name

An example is presented:

```
JSON
{
  "to": "user@example.com",
  "subject": "User Registered",
  "type": "SUCCESSFUL_REGISTER",
  "params": {
    "name": "John Doe"
  }
}
```

b. PASSWORD_RESET

Param	Type	Description
`name`	string	User's name
`token`	string	Password reset code

An example is presented:

```
JSON
{
  "to": "user@example.com",
  "subject": "Reset your password",
  "type": "PASSWORD_RESET",
  "params": {
    "name": "John Doe",
    "token": "823912"
  }
}
```

c. EVENT_CONFIRMATION

Param	Type	Description

`name`	string	User's name
`eventName`	string	Event name
`eventDate`	string	Event date
`location`	string	Event location

An example is presented:

```JSON
{
  "to": "user@example.com",
  "subject": "You're confirmed!",
  "type": "EVENT_CONFIRMATION",
  "params": {
    "name": "John Doe",
    "eventName": "Tech Summit",
    "eventDate": "2025-05-22",
    "location": "Main Auditorium"
  }
}
```

d. WAITLIST_NOTIFICATION

Param	Type	Description
`name`	string	User's name
`eventName`	string	Event name

An example is presented:

```JSON
{
  "to": "user@example.com",
  "subject": "You're on the waitlist",
  "type": "WAITLIST_NOTIFICATION",
  "params": {
    "name": "John Doe",
```

```
    "eventName": "Tech Summit"
  }
}
```

e. WAITLIST_PROMOTION

Param	Type	Description
`name`	string	User's name
`eventName`	string	Event name
`eventDate`	string	Event date
`location`	string	Event location

An example is presented:

```JSON
{
  "to": "user@example.com",
  "subject": "A spot opened for you!",
  "type": "WAITLIST_PROMOTION",
  "params": {
    "name": "John Doe",
    "eventName": "Tech Summit",
    "eventDate": "2025-06-01",
    "location": "Conference Room B"
  }
}
```

f. CAPACITY_REACHED

Param	Type	Description
`name`	string	Admin or responsible user’s name
`eventName`	string	Event name

`capacity`	number	Maximum attendee capacity reached

An example is presented:

```json
{
  "to": "admin@example.com",
  "subject": "Event capacity reached",
  "type": "CAPACITY_REACHED",
  "params": {
    "name": "Admin",
    "eventName": "Tech Summit",
    "capacity": 150
  }
}
```